

□ | *logo.jpg*

Smart Parking Access Control (SPAC)

An End-to-End License Plate Recognition System for Automated Vehicle Authorization

Arad Fadaei, Sia Tedy

Table of Contents

1. Introduction	2
1.1. Problem Statement	2
1.2. Motivation	2
1.3. Objectives	2
2. Project Repository & Tooling	3
2.1. Repository Structure	3
2.2. CLI Workflow Runner	3
2.3. Centralized Configuration	4
2.4. Additional Tooling	5
3. Background & Related Work	6
3.1. Automatic License Plate Recognition	6
3.2. Object Detection with YOLO	6
3.3. Optical Character Recognition	6
3.4. String Matching for Plate Verification	6
4. System Architecture	7
4.1. Pipeline Stages	7
5. Methodology	8
5.1. Dataset & Preparation	8
5.1.1. Source Data	8
5.1.2. Data Conversion	8
5.1.3. Resident Database	8
5.2. Plate Detection	8
5.2.1. Model Selection	8
5.2.2. Training Configuration	8
5.2.3. Inference Configuration	9
5.3. OCR Methodology	9
5.3.1. OCR Design Rationale	9
5.3.2. Preprocessing Strategy	9
5.3.3. OCR Backends and Text Extraction	10
5.3.4. Candidate Ranking and Confidence Handling	11
5.3.5. Example Preprocessing Outputs	11
5.4. Resident Matching	12
5.4.1. Text Normalization	12
5.4.2. Matching Strategy	12
5.4.3. Decision Logic	12
6. Evaluation	14
6.1. Evaluation Methodology	14
6.1.1. Base Metrics (No Ground Truth Required)	14

6.1.2. Ground-Truth Metrics (Labeled Data Required)	14
6.2. Results	14
6.2.1. Base Pipeline Metrics	14
6.2.2. Decision Metrics (Ground-Truth Evaluation)	15
6.2.3. Confusion Matrix	15
6.2.4. Plate Exact-Match Rate	15
6.3. Analysis & Discussion	15
6.3.1. Strengths	15
6.3.2. Weaknesses	15
6.3.3. False Positive Analysis	16
7. Preprocessing Visualization	17
8. Limitations & Future Work	18
8.1. Current Limitations	18
8.2. Future Work	18
9. Conclusion	19
References	20

Abstract

This report presents the design, implementation, and evaluation of Smart Parking Access Control (SPAC), an end-to-end computer vision system that automates vehicle authorization at parking facilities. The system integrates YOLOv8 object detection for license plate localization, a dual-backend OCR engine with ten preprocessing variants for robust text extraction, and a fuzzy-matching module for resident database lookup. Evaluated on 44 test images from the Kaggle Car Plate Detection dataset, the system achieves a 97.7% plate detection rate, 88.6% decision accuracy, and 85.7% recall for authorized residents. The modular pipeline is fully configurable via a centralized YAML configuration and accessible through a unified command-line interface supporting all workflow stages from data preparation through evaluation.

1. Introduction

1.1. Problem Statement

Parking access control remains a labor-intensive process at many residential and commercial facilities. Traditional approaches rely on physical access cards, keypads, or manned security checkpoints — all of which incur ongoing operational costs, introduce human error, and inconvenience authorized residents. An automated vision-based system that can identify vehicles by their license plates and grant or deny access in real time offers a compelling alternative.

1.2. Motivation

Automatic License Plate Recognition (ALPR) has matured significantly in recent years, driven by advances in deep learning-based object detection and optical character recognition. However, assembling a reliable, end-to-end ALPR pipeline for access control requires careful integration of multiple components — detection, OCR, text normalization, and database matching — each with their own failure modes. This project explores how these components can be composed into a practical, configurable system with measurable performance.

1.3. Objectives

The primary objectives of this project are:

1. **Detection:** Train a lightweight object detector to reliably localize license plates in images.
2. **Recognition:** Extract plate text using multi-backend OCR with robust preprocessing.
3. **Matching:** Match extracted text against an authorized resident database using exact and fuzzy matching.
4. **Decision:** Produce access grant/deny decisions with confidence scores.
5. **Evaluation:** Quantify system performance against ground-truth labels.
6. **Tooling:** Provide a reproducible, configurable pipeline accessible via a single CLI.

2. Project Repository & Tooling

This section describes the repository structure, the CLI workflow runner, and the tooling built to support reproducible experimentation.

2.1. Repository Structure

The project follows a modular source layout with clear separation between data, training, inference, evaluation, and utility code.

```
CV_SPAC/
├── configs/
│   └── default.yaml          # Centralized configuration
├── data/
│   ├── db/
│   │   └── residents.csv    # Resident database
│   ├── processed/          # YOLO-format dataset (gitignored)
│   └── raw/                 # Raw Kaggle download (gitignored)
├── docs/
│   └── evaluation_tables.md # Auto-generated evaluation report
├── models/
│   └── plate_detector.pt    # Trained YOLO weights
├── outputs/
│   ├── demo/              # Annotated output images
│   ├── metrics/           # Evaluation JSON, CSV, ground truth
│   ├── predictions/       # Inference results JSON
│   └── preprocessing_steps/ # Per-image OCR preprocessing exports
├── runs/                  # YOLO training artifacts
├── src/
│   ├── main.py            # CLI entrypoint & workflow runner
│   ├── data/              # Dataset preparation & DB seeding
│   ├── train/             # Detector training module
│   ├── infer/             # Inference pipeline & preprocessing export
│   ├── eval/              # Evaluation & report generation
│   ├── ocr/               # Multi-backend OCR engine
│   ├── matching/          # Resident plate matcher
│   └── utils/              # Shared text normalization helpers
├── requirements.txt
└── README.md
```

2.2. CLI Workflow Runner

All pipeline stages are accessible through a single entrypoint (`src.main`) that reads configuration from `configs/default.yaml` and dispatches to the appropriate module based on command-line flags.

Table 1. CLI Flags and Pipeline Stages

Flag	Stage	Module
<code>--download</code>	Download and unzip Kaggle dataset	Kaggle CLI
<code>--seed-db</code>	Generate sample resident database	<code>src.data.create_sample_db</code>
<code>--prepare</code>	Convert raw XML annotations to YOLO format	<code>src.data.prepare_kaggle_car_plate_dataset</code>
<code>--train</code>	Train plate detector	<code>src.train.train_detector</code>
<code>--infer</code>	Run end-to-end inference pipeline	<code>src.infer.run_inference</code>
<code>--eval</code>	Evaluate predictions against ground truth	<code>src.eval.evaluate_pipeline</code>
<code>--report</code>	Generate formatted evaluation tables	<code>src.eval.generate_report_tables</code>
<code>--all</code>	Run all stages sequentially	—

Stages can be composed freely:

```
# Run inference, evaluation, and report generation
python -m src.main --infer --eval --use-ground-truth --report

# Full pipeline end-to-end
python -m src.main --all

# Single image inference
python -m src.main --infer --image path/to/image.jpg
```

2.3. Centralized Configuration

All paths, thresholds, hyperparameters, and behavioral flags are defined in a single YAML configuration file (`configs/default.yaml`), eliminating hardcoded values and enabling reproducible experiments.

Table 2. Key Configuration Sections

Section	Purpose
<code>paths</code>	File paths for database, outputs, weights, and reports
<code>dataset</code>	Download directory, train/val/test split ratios (80/10/10), random seed
<code>training</code>	Model architecture (YOLOv8n), epochs (80), image size (640), batch size (16)
<code>inference</code>	Detection confidence threshold (0.25), OCR threshold (0.35), fuzzy match threshold (90%)
<code>ocr</code>	Backend selection (EasyOCR, PaddleOCR), language, rectification toggle
<code>evaluation</code>	Positive label for decision metrics ("Access Granted")

2.4. Additional Tooling

Preprocessing Steps Export: A standalone tool (`src.infer.export_preprocessing_steps`) saves intermediate preprocessing images for any input, enabling visual debugging of the OCR pipeline.

```
python -m src.infer.export_preprocessing_steps --image
data/processed/car_plate_kaggle/images/test/Cars111.png
```

Ground Truth Template Generator: Automatically creates a CSV template from inference results for manual annotation (`src.eval.create_ground_truth_template`).

Report Table Generator: Produces formatted Markdown evaluation tables from raw metrics JSON (`src.eval.generate_report_tables`).

3. Background & Related Work

3.1. Automatic License Plate Recognition

Automatic License Plate Recognition (ALPR) systems have been widely studied in the computer vision literature. A typical ALPR pipeline consists of three stages: plate detection (localization), character segmentation, and character recognition. Modern deep learning approaches often collapse segmentation and recognition into a single OCR step, simplifying the pipeline to detection followed by end-to-end text recognition.

3.2. Object Detection with YOLO

The YOLO (You Only Look Once) family of detectors [redmon2016] has become a standard choice for real-time object detection tasks. YOLOv8 [jocher2023], developed by Ultralytics, provides a range of model sizes from nano (YOLOv8n, 3.2M parameters) to extra-large. The nano variant offers favorable speed-accuracy tradeoffs for edge deployment, making it suitable for real-time plate detection where inference latency matters.

3.3. Optical Character Recognition

Modern OCR engines such as EasyOCR [jaided2020] and PaddleOCR [du2020] leverage deep neural networks for text detection and recognition. EasyOCR uses a CRAFT-based text detector with a sequence-to-sequence recognizer, while PaddleOCR employs a PP-OCR pipeline with lightweight models. Both support multiple languages and are available as open-source Python libraries.

The quality of OCR results is highly sensitive to image preprocessing. Common techniques include grayscale conversion, histogram equalization (CLAHE), binarization (Otsu’s method, adaptive thresholding), denoising, and geometric rectification — all of which are employed in this system.

3.4. String Matching for Plate Verification

Exact string matching is insufficient for plate verification due to OCR character confusions (e.g., 0 ↔ O, 1 ↔ I, 8 ↔ B). Fuzzy string matching algorithms such as the Levenshtein distance and its normalized variants provide a similarity score that can tolerate minor character errors. RapidFuzz [maxbachmann2021] is a high-performance Python library implementing these algorithms, suitable for real-time matching.

4. System Architecture

The SPAC system is organized as a four-stage sequential pipeline, where each stage is an independent, replaceable module.



System Architecture Overview

Placeholder for architecture diagram.

Insert a figure showing the pipeline flow: Input Image → Plate Detection → Plate Cropping → OCR Engine (with preprocessing variants) → Text Normalization → Resident Matching → Access Decision.

4.1. Pipeline Stages

1. **Plate Detection** — A YOLOv8n model localizes the license plate region within the input image, returning a bounding box and confidence score.
2. **Plate Cropping** — The detected region is extracted with configurable padding (14% horizontal, 22% vertical) to preserve plate context while excluding background noise.
3. **OCR Text Extraction** — A dual-backend OCR engine processes the cropped plate through ten or more preprocessing variants, scoring candidates by OCR confidence and a plausibility heuristic.
4. **Resident Matching** — The extracted text is normalized and compared against a resident database using exact matching followed by fuzzy matching (90% similarity threshold).

Each stage produces structured output (bounding box, text, match result), which is aggregated into a per-image result record and serialized to JSON for downstream evaluation.

5. Methodology

5.1. Dataset & Preparation

5.1.1. Source Data

The system uses the **Kaggle Car Plate Detection** dataset ([andrewmvd/car-plate-detection](#)) consisting of 433 annotated vehicle images with license plate bounding boxes in Pascal VOC XML format.

5.1.2. Data Conversion

The preparation module (`src.data.prepare_kaggle_car_plate_dataset`) performs:

1. **XML Parsing:** Extracts bounding box coordinates (`xmin`, `ymin`, `xmax`, `ymax`) from VOC-format annotations.
2. **YOLO Format Conversion:** Converts absolute pixel coordinates to YOLO center-normalized format: (`x_center`, `y_center`, `width`, `height`) relative to image dimensions.
3. **Train/Val/Test Split:** Stratified random split with ratios 80/10/10 (seed 42), producing 346 training, 43 validation, and 44 test images.
4. **Dataset YAML:** Generates an Ultralytics-compatible `dataset.yaml` specifying paths and class names.

5.1.3. Resident Database

A synthetic resident database (`data/db/residents.csv`) is seeded from ground-truth plate values and OCR predictions. The database contains 36 entries with fields: `plate_number`, `resident_name`, `unit`, `vehicle_color`, and `status` (`active` or `inactive`). The `active_ratio` of 0.85 produces approximately 31 active and 5 inactive records, simulating a realistic access control scenario.

5.2. Plate Detection

5.2.1. Model Selection

YOLOv8n (nano) was selected as the detection backbone for its balance of speed and accuracy. With 3.2 million parameters and a model size of approximately 6 MB, it is suitable for deployment on resource-constrained hardware.

5.2.2. Training Configuration

Table 3. Detector Training Hyperparameters

Parameter	Value
Base Model	<code>yolov8n.pt</code> (COCO pretrained)
Epochs	80
Image Size	640 × 640

Parameter	Value
Batch Size	16
Device	GPU (CUDA device 0)
Optimizer	SGD (Ultralytics default)

The model was fine-tuned on the prepared dataset with a single class (plate). Training artifacts — including weights, loss curves, and per-epoch metrics — are saved to `runs/detect/outputs/detector_train/`.

5.2.3. Inference Configuration

At inference time, the detector runs with a confidence threshold of 0.25. The highest-confidence detection per image is selected, and the corresponding bounding box is passed to the cropping stage. If no detection exceeds the threshold, the image is marked as "no detection" and the pipeline skips OCR and matching for that sample.

5.3. OCR Methodology

The OCR stage is responsible for converting a detected plate crop into a normalized alphanumeric string that can be matched against the resident database. This stage is intentionally more redundant than the rest of the pipeline because OCR quality is highly sensitive to blur, plate skew, contrast variation, specular reflections, and small character size. Rather than committing to a single preprocessing routine or a single recognizer, the system generates multiple plate representations and evaluates them with two OCR backends before selecting the most plausible result.

5.3.1. OCR Design Rationale

The detector returns only a localized crop, not a fronto-parallel or uniformly illuminated plate image. In practice, many failures arise from geometric distortion and local contrast loss rather than complete detector failure. For that reason, the OCR stage was designed around three principles: improve legibility through preprocessing, diversify recognition using multiple engines, and delay final selection until all candidates have been scored.

This design also keeps the OCR component modular. Each preprocessing operation is implemented as a self-contained step, and the sequential pipeline order can be modified without changing the rest of the inference code. Likewise, the backend list is configurable through `configs/default.yaml`, allowing the OCR stage to be simplified or expanded without altering the surrounding pipeline.

5.3.2. Preprocessing Strategy

The OCR engine begins from the padded plate crop produced by the inference pipeline. That crop already preserves some surrounding context by extending the detector bounding box by 14% horizontally and 22% vertically, which reduces the risk of clipping border characters. The OCR module then generates several derived images intended to address common visual failure modes.

Table 4. OCR Preprocessing Operations

Operation	Purpose
Upscaling	Magnifies small plate crops by 2x or 3x using bicubic interpolation so thin strokes become easier to resolve.
Rectification	Estimates the dominant plate orientation from the largest contour and rotates the crop to reduce in-plane skew.
Perspective Flattening	Approximates the plate outline as a quadrilateral and applies a perspective warp to reduce projective distortion.
Grayscale Conversion	Removes color information when it is not useful for recognition and simplifies later enhancement steps.
Bilateral Denoising	Reduces local noise while preserving edges that define character boundaries.
CLAHE	Improves local contrast under uneven illumination without amplifying the entire image uniformly.
Otsu Binarization	Produces a global black-and-white separation when character strokes are already reasonably distinct.
Adaptive Thresholding	Uses local Gaussian thresholding to handle plates with non-uniform lighting or shadows.
Sharpening	Applies a simple enhancement kernel to strengthen stroke edges before recognition.

In addition to these individual variants, the engine also evaluates a sequential preprocessing pipeline whose default order is:

```
upscale -> rectify -> flatten -> grayscale -> denoise -> clahe -> sharpen -> binarize
```

This arrangement reflects a practical assumption: geometric corrections should occur before contrast enhancement and thresholding, while binary conversion should occur late in the process after noise and contrast have already been addressed. The engine does not assume that this sequence is always optimal, which is why the individual variants are preserved and evaluated alongside the full sequential version.

5.3.3. OCR Backends and Text Extraction

Two recognizers are used: EasyOCR and PaddleOCR. EasyOCR is configured with an uppercase alphanumeric allowlist and beam-search decoding, making it well suited to constrained plate text. PaddleOCR serves as a complementary recognizer with a different detection and recognition stack, which helps recover cases where one backend fragments the text or misreads individual characters.

For each image variant, the engine queries both backends and collects multiple candidate strings. When a backend produces several text boxes, the system keeps both the single highest-confidence box and a left-to-right concatenation of all detected boxes. This is important for license plates because characters are sometimes split across multiple OCR regions even though they form one logical identifier.

After recognition, all candidate strings are normalized before ranking. Normalization converts characters to uppercase, strips whitespace, and removes non-alphanumeric symbols. This makes downstream comparison more stable and prevents punctuation or spacing artifacts from being treated as meaningful differences.

5.3.4. Candidate Ranking and Confidence Handling

The final OCR output is selected from the full pool of backend and preprocessing candidates rather than from a single recognizer pass. Each candidate is ranked by a combination of OCR confidence and a lightweight plate-plausibility heuristic. The heuristic rewards strings whose lengths resemble typical plate formats, gives a small bonus to mixed letter-digit strings, and penalizes extremely short outputs or repetitive low-diversity strings that are more likely to be OCR noise.

Formally, candidates are ordered by a tuple of the form (confidence + plausibility, confidence, text_length). This ranking strategy is strong enough to suppress obviously poor reads while remaining simple to explain and modify. Because it is applied only at the selection stage, it can be removed or replaced later without changing the rest of the recognition pipeline.

The OCR confidence threshold in the configuration should be interpreted as a control parameter rather than as a strict rejection gate. In the current implementation, low-confidence text can still be propagated for inspection and downstream evaluation, which is useful during error analysis even when the read would not be trusted in a production system. This behavior reflects the exploratory nature of the project and keeps failure cases visible instead of silently discarding them.

5.3.5. Example Preprocessing Outputs

To make the OCR stage interpretable, the project includes an export utility that saves intermediate images for a selected sample. The example below uses **Cars111**, which produces the final OCR output **MH20EE7598** with confidence **0.997474**.

Table 5. Cars111 Source Images

image:.../outputs/preprocessing_steps/Cars111/01_original.png[original input, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/02_detected_bbox.png[detected bounding box, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/03_plate_crop.png[padded plate crop, pdfwidth=1.55in]
--	--	---

Table 6. Cars111 Geometric Variants

image:.../outputs/preprocessing_steps/Cars111/04_upscaled.png[upscaled crop, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/05_rectified_or_upscaled.png[rectified crop, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/06_flattened.png[flattened crop, pdfwidth=1.55in]
---	---	---

Table 7. Cars111 Enhancement Variants I

image:.../outputs/preprocessing_steps/Cars111/07_denoise.png[denoised crop, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/08_clahe.png[clahe crop, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/09_otsu_threshold.png[otsu threshold, pdfwidth=1.55in]
--	---	--

Table 8. Cars111 Enhancement Variants II

image:.../outputs/preprocessing_steps/Cars111/10_adaptive_threshold.png[adaptive threshold, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/11_sharpen.png[s harpened crop, pdfwidth=1.55in]	image:.../outputs/preprocessing_steps/Cars111/12_rectified_gray.png[rectified grayscale crop, pdfwidth=1.55in]
--	--	--

[sequential pipeline output] | ../outputs/preprocessing_steps/Cars111/13_sequential_pipeline.png
Figure 1. Cars111 Sequential Pipeline Output

Cars111 OCR Export Summary

```
image: data/processed/car_plate_kaggle/images/test/Cars111.png
bbox_xyxy: (323, 162, 476, 205)
detector_conf: 0.817018
final_ocr_text: MH20EE7598
final_ocr_conf: 0.997474
```

Together, these artifacts provide a visual explanation of how the OCR stage transforms a detector crop into a final text prediction. They are also useful for debugging because they expose which preprocessing variants preserve character structure and which ones over-process the image.

5.4. Resident Matching

5.4.1. Text Normalization

Before matching, plate text is normalized:

- Converted to uppercase
- Non-alphanumeric characters removed
- Whitespace stripped

This ensures consistent comparison regardless of OCR formatting artifacts.

5.4.2. Matching Strategy

The matcher employs a two-tier approach:

1. **Exact Match:** The normalized plate is looked up directly in a precomputed dictionary of normalized resident plates. If found, the match score is 100%.
2. **Fuzzy Match:** If exact matching fails and fuzzy matching is enabled, RapidFuzz computes the normalized Levenshtein similarity between the input plate and every resident plate. The best match is returned if its score meets or exceeds the threshold (default: 90%).

5.4.3. Decision Logic

Table 9. Access Decision Matrix

Condition	Status	Decision
Match $\geq$ threshold, active status	Active	Access Granted
Match $\geq$ threshold, inactive status	Inactive	Access Denied
No match or score $<$ threshold	—	Access Denied

6. Evaluation

6.1. Evaluation Methodology

The evaluation module computes metrics at two levels:

6.1.1. Base Metrics (No Ground Truth Required)

- **Detection rate:** Fraction of images where the detector found a plate.
- **OCR success rate:** Fraction of detected plates that produced non-empty text.
- **Decision distribution:** Count of granted vs. denied decisions.
- **Mean confidences:** Average detector and OCR confidence scores.

6.1.2. Ground-Truth Metrics (Labeled Data Required)

With a manually annotated ground-truth CSV specifying expected decisions and plate text per image:

- **Decision metrics:** Accuracy, Precision, Recall, F1-Score computed by treating "Access Granted" as the positive class.
- **Confusion matrix:** True Positives, True Negatives, False Positives, False Negatives.
- **Plate exact-match rate:** Fraction of plates where OCR output exactly matches the ground-truth plate text (excluding samples labeled "UNKNOWN").

6.2. Results

6.2.1. Base Pipeline Metrics

Table 10. Base Pipeline Performance (44 Test Images)

Metric	Value
Total Samples	44
Plates Detected	43 / 44
Detection Rate	97.73%
OCR Non-Empty	43 / 44
OCR Success Rate	97.73%
Predicted Granted	10
Predicted Denied	34
Mean Detector Confidence	0.761
Mean OCR Confidence	0.593

6.2.2. Decision Metrics (Ground-Truth Evaluation)

Table 11. Decision Classification Metrics

Metric	Value
Labeled Samples	44
Accuracy	88.64%
Precision	60.00%
Recall	85.71%
F1-Score	70.59%

6.2.3. Confusion Matrix

Table 12. Confusion Matrix (Positive = "Access Granted")

	Predicted Granted	Predicted Denied
Actually Granted	TP = 6	FN = 1
Actually Denied	FP = 4	TN = 33

6.2.4. Plate Exact-Match Rate

Table 13. Plate Text Recognition Accuracy

Metric	Value
Labeled Plate Samples	39
Exact Match Rate	30.77% (12 / 39)

6.3. Analysis & Discussion

6.3.1. Strengths

High Detection Reliability. The YOLOv8n detector achieves a 97.73% detection rate, failing on only 1 of 44 test images. This demonstrates that even the nano-sized model, when fine-tuned on domain-specific data, provides robust plate localization.

Strong Recall. The system correctly identifies 6 of 7 authorized residents (85.71% recall), meaning it rarely denies access to legitimate residents. In an access control scenario, high recall is critical for user satisfaction.

Effective Fuzzy Matching. Despite a low plate exact-match rate (30.77%), fuzzy matching at the 90% threshold compensates for OCR character errors. The gap between plate exact-match (30.8%) and decision accuracy (88.6%) illustrates the value of the fuzzy matching layer.

6.3.2. Weaknesses

Low Precision (60%). Of 10 images predicted as "Access Granted," 4 are false positives —

unauthorized vehicles granted access. In a security context, this is the most concerning metric. Root causes include:

- Fuzzy matching accepting plates with coincidental similarity to a resident entry
- OCR character confusions (e.g., 0 → 0, 1 → I) producing text that closely resembles a different resident's plate

Low Plate Exact-Match Rate (30.8%). Only 12 of 39 labeled plates were read with perfect character accuracy. Contributing factors include:

- Variable image quality and lighting conditions
- Plate designs with decorative fonts or non-standard layouts
- Small plate regions requiring aggressive upscaling
- Perspective distortions not fully corrected by rectification

Moderate OCR Confidence (0.593 mean). The average OCR confidence is below 0.60, indicating the engine operates near its reliability boundary on this dataset. Confidence-based filtering mitigates but does not eliminate unreliable reads.

6.3.3. False Positive Analysis

The 4 false positives are the primary area for improvement. Potential mitigations include:

- Raising the fuzzy match threshold (e.g., from 90% to 95%)
- Requiring a minimum OCR confidence for access decisions
- Implementing a confirmation mechanism (e.g., second image capture)
- Adding temporal consistency (same plate recognized across multiple frames)

7. Preprocessing Visualization

The preprocessing export tool enables visual inspection of each OCR variant for a given input image. [Preprocessing Steps Export](#) illustrates the intermediate outputs for a sample plate.



Preprocessing Steps Export

Placeholder for preprocessing visualization images.

Insert a figure grid showing the preprocessing outputs from [outputs/preprocessing_steps/](#) — original crop, upscaled, rectified, flattened, grayscale, denoised, CLAHE, binarized, adaptive threshold, sharpened, and full pipeline variants.

Each variant is saved as a separate image file alongside a [summary.txt](#) describing the OCR output, confidence, and backend used for that variant. This tooling facilitates rapid diagnosis of OCR failures and guides preprocessing tuning.

8. Limitations & Future Work

8.1. Current Limitations

1. **Small Test Set:** Evaluation on 44 images provides indicative but not statistically conclusive results. A larger, more diverse dataset would strengthen confidence in the reported metrics.
2. **Single Frame Decisions:** Each image is processed independently. Real-world deployments would benefit from temporal smoothing across multiple frames of the same vehicle.
3. **Synthetic Resident Database:** The resident database is auto-generated from ground-truth and OCR predictions rather than representing a genuine facility database.
4. **Static Thresholds:** Confidence and fuzzy matching thresholds are fixed globally. Adaptive thresholds based on image quality could improve performance.
5. **No Plate Segmentation:** The system detects the plate region but does not segment individual characters. Character-level recognition could improve accuracy for plates with uncommon layouts.

8.2. Future Work

1. **Threshold Optimization:** Systematic sweep of detector, OCR, and fuzzy matching thresholds to find the Pareto-optimal operating point for the precision–recall tradeoff.
2. **Data Augmentation:** Augmenting training data with synthetic distortions (blur, noise, perspective) to improve detector and OCR robustness.
3. **Multi-Frame Fusion:** Aggregating detections across video frames using temporal voting or plate-tracking to reduce single-frame errors.
4. **Specialized OCR Models:** Fine-tuning a recognition model specifically on license plates (e.g., using STR benchmarks) rather than relying on general-purpose OCR.
5. **Edge Deployment:** Profiling and optimizing the pipeline for embedded hardware (Jetson Nano, Raspberry Pi) using TensorRT or ONNX Runtime.
6. **Expanded Evaluation:** Testing on diverse plate standards (European, Asian formats) and under controlled degradation conditions (rain, low light, motion blur).

9. Conclusion

This project demonstrates a practical, modular approach to automated parking access control using license plate recognition. The SPAC system integrates YOLOv8 detection, dual-backend OCR with extensive preprocessing, and fuzzy string matching into a configurable pipeline that achieves 88.6% decision accuracy and 85.7% recall on its test set.

The modular architecture — with each pipeline stage independently replaceable — supports iterative improvement and experimentation. The centralized YAML configuration and unified CLI eliminate manual configuration and enable reproducible workflows.

Key findings include the critical role of fuzzy matching in bridging the gap between imperfect OCR output (30.8% exact match) and practical access decisions (88.6% accuracy), as well as the importance of multi-variant preprocessing for maximizing OCR robustness under diverse image conditions.

The primary area for improvement is precision (60%), where false positive access grants pose security risks. Future work should focus on threshold optimization, multi-frame fusion, and specialized OCR models to close this gap.

References

- [redmon2016] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.
- [jocher2023] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0). <https://github.com/ultralytics/ultralytics>
- [jaided2020] JaidedAI. (2020). EasyOCR: Ready-to-use OCR with 80+ supported languages. <https://github.com/JaidedAI/EasyOCR>
- [du2020] Du, Y., et al. (2020). PP-OCR: A Practical Ultra Lightweight OCR System. *arXiv preprint arXiv:2009.09941*.
- [maxbachmann2021] Bachmann, M. (2021). RapidFuzz: Rapid fuzzy string matching in Python. <https://github.com/maxbachmann/RapidFuzz>
- [opencv] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [otsu1979] Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62–66.